

Introducción al Aprendizaje Automático

Práctica 9: Aprendizaje con Pocos Ejemplos

Sebastián Ventura Christian Luna
sventura@uco.es c.luna@uco.es

3er Curso Grado en Ingeniería Informática
Especialidad Computación
Escuela Politécnica Superior
(Universidad de Córdoba)

20 de mayo de 2026



Objetivos de esta parte teórica

Qué necesitamos entender

- Qué significa aprender cuando hay **muy pocos ejemplos etiquetados**.
- Por qué no siempre podemos entrenar un clasificador convencional desde cero.
- Cómo reutilizar una red entrenada como **extractor de características**.
- Qué son el **support set**, el **query set** y los **prototipos**.
- Cómo clasificar nuevas imágenes mediante **distancias entre embeddings**.

Si solo tenemos 1 o 5 ejemplos de una clase nueva, ¿podemos reconocer nuevas imágenes de esa clase?

Objetivos de esta parte teórica

Qué necesitamos entender

- Qué significa aprender cuando hay **muy pocos ejemplos etiquetados**.
- Por qué no siempre podemos entrenar un clasificador convencional desde cero.
- Cómo reutilizar una red entrenada como **extractor de características**.
- Qué son el **support set**, el **query set** y los **prototipos**.
- Cómo clasificar nuevas imágenes mediante **distancias entre embeddings**.

Si solo tenemos 1 o 5 ejemplos de una clase nueva, ¿podemos reconocer nuevas imágenes de esa clase?

Idea clave

Few-shot Learning no consiste simplemente en entrenar con pocos datos. La clave está en **reutilizar una representación aprendida previamente**.

El problema: aprender con pocos ejemplos

Situación habitual

En muchos problemas reales no tenemos miles de ejemplos etiquetados para cada clase.

Ejemplos

- una nueva categoría visual;
- un defecto raro en producción;
- una enfermedad poco frecuente;
- una especie difícil de observar;
- un nuevo tipo de documento, imagen o señal.

Riesgo

Con tan pocos datos, entrenar un modelo desde cero puede llevar a memorizar ejemplos concretos en lugar de aprender una regla general.

Pregunta

¿Cómo aprovechamos lo que un modelo ya ha aprendido antes?

Clasificación convencional vs Few-shot Learning

Clasificación supervisada clásica

- muchas muestras por clase;
- entrenamiento directo del clasificador;
- la última capa separa las clases conocidas;
- si aparece una clase nueva, suele requerir reentrenamiento.

Few-shot Learning

- pocas muestras de la nueva clase;
- se reutiliza una representación aprendida;
- la nueva clase se define con ejemplos de apoyo;
- la decisión se basa en similitud o distancia.

Diferencia fundamental

No queremos aprender toda la clase desde cero. Queremos **representarla bien con pocos ejemplos**.

Clasificación convencional vs Few-shot Learning

Clasificación supervisada clásica

- muchas muestras por clase;
- entrenamiento directo del clasificador;
- la última capa separa las clases conocidas;
- si aparece una clase nueva, suele requerir reentrenamiento.

Few-shot Learning

- pocas muestras de la nueva clase;
- se reutiliza una representación aprendida;
- la nueva clase se define con ejemplos de apoyo;
- la decisión se basa en similitud o distancia.

Diferencia fundamental

No queremos aprender toda la clase desde cero. Queremos **representarla bien con pocos ejemplos**.

Intuición

No es: “entrena un modelo nuevo con 5 imágenes”. Es: “usa esas 5 imágenes para localizar esa clase en el espacio de características”.

La idea de esta práctica: el mundo sin sietes

Escenario experimental

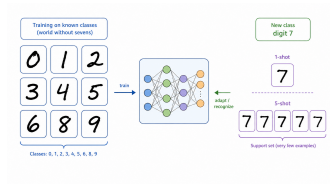
Entrenaremos inicialmente un modelo con MNIST, pero eliminando todas las imágenes del dígito **7** durante el entrenamiento.

Clases conocidas

0, 1, 2, 3, 4, 5, 6, 8, 9

Clase nueva

Después aparecerá el dígito **7**, pero solo tendremos 1 o 5 ejemplos etiquetados para representarlo.



El modelo aprende un mundo donde el 7 todavía no existe.

Por qué eliminar el 7 es importante

Motivo

Queremos simular una situación realista: aparece una clase nueva después de haber entrenado el modelo.

Si entrenamos con el 7

El problema sería una clasificación convencional de MNIST.

Si no entrenamos con el 7

Podemos estudiar si el modelo reutiliza lo aprendido para representar una clase nueva.

Mensaje importante

El modelo no aprende explícitamente la clase 7, pero sí puede aprender rasgos visuales útiles: trazos, bordes, curvas, inclinaciones y formas parciales.

Fase 1: entrenar un clasificador base

Qué hacemos primero

Entrenamos una red neuronal sencilla para clasificar los dígitos conocidos.

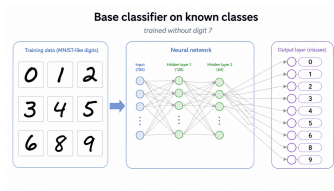
Objetivo inicial

No buscamos todavía reconocer el 7.
Buscamos que la red aprenda una representación útil de imágenes de dígitos.

Salida del clasificador

La última capa solo distingue entre:

0, 1, 2, 3, 4, 5, 6, 8, 9



La red se entrena como clasificador convencional.

Fase 2: convertir el modelo en extractor

Cambio de uso

Una vez entrenado el clasificador, eliminamos o ignoramos la última capa de clasificación.

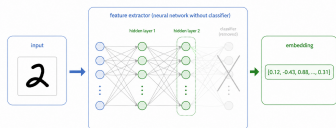
Qué conservamos

Conservamos las capas que transforman la imagen en una representación interna.

Embedding

Cada imagen deja de verse como una matriz de píxeles y pasa a representarse como un vector numérico:

$$f(x) \in \mathbb{R}^d$$



La imagen se transforma en un vector de características.

Por qué no comparar directamente píxeles

Comparar píxeles parece sencillo

Podríamos medir la distancia entre dos imágenes usando directamente sus valores de píxel.

Problema

Dos imágenes del mismo dígito pueden tener píxeles bastante distintos:

- diferente inclinación;
- diferente grosor del trazo;
- distinta posición;
- distinta forma de escribir.

Mejor alternativa

Comparar embeddings permite trabajar con una representación más semántica y menos dependiente de cada píxel concreto.

Intuición

Dos siete escritos de forma distinta pueden estar lejos en píxeles, pero cerca en un buen espacio de características.

Support set y query set

Support set

Pequeño conjunto de ejemplos etiquetados que usamos para definir o representar una clase.

En esta práctica

- 1 ejemplo para **1-shot**;
- 5 ejemplos para **5-shot**.

Query set

Conjunto de ejemplos que queremos clasificar una vez construidos los prototipos.

En esta práctica

Usaremos imágenes de test para comprobar si el sistema clasifica correctamente usando distancias.

Idea clave

El support set define las clases; el query set permite evaluar si esa definición funciona.

Support set y query set

Support set

Pequeño conjunto de ejemplos etiquetados que usamos para definir o representar una clase.

En esta práctica

- 1 ejemplo para **1-shot**;
- 5 ejemplos para **5-shot**.

Query set

Conjunto de ejemplos que queremos clasificar una vez construidos los prototipos.

En esta práctica

Usaremos imágenes de test para comprobar si el sistema clasifica correctamente usando distancias.

Idea clave

El support set define las clases; el query set permite evaluar si esa definición funciona.

Pregunta para pensar

¿Qué ocurre si los pocos ejemplos del support set no representan bien la variabilidad real de la clase?

Qué es un prototipo

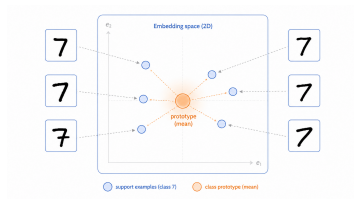
Idea intuitiva

Un prototipo es una representación media de una clase en el espacio de embeddings.

Cómo se calcula

Tomamos los ejemplos del support set, obtenemos sus embeddings y calculamos el vector medio.

$$c_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} f(x_i)$$



El prototipo resume la posición de una clase.

Qué es un prototipo

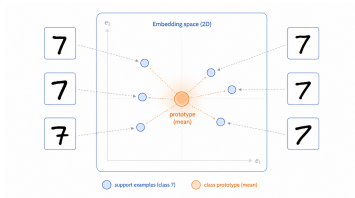
Idea intuitiva

Un prototipo es una representación media de una clase en el espacio de embeddings.

Cómo se calcula

Tomamos los ejemplos del support set, obtenemos sus embeddings y calculamos el vector medio.

$$c_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} f(x_i)$$



El prototipo resume la posición de una clase.

Mensaje importante

El prototipo no es una imagen real. Es un punto medio en el espacio de características.

1-shot frente a 5-shot

1-shot

La clase se representa usando un único ejemplo.

Ventaja

Es el caso más extremo y barato: solo necesitamos una muestra etiquetada.

Riesgo

Ese único ejemplo puede no ser representativo.

5-shot

La clase se representa usando cinco ejemplos.

Ventaja

El prototipo suele ser más estable porque promedia varias muestras.

Riesgo

Sigue habiendo pocos datos, así que la representación puede ser limitada.

1-shot frente a 5-shot

1-shot

La clase se representa usando un único ejemplo.

Ventaja

Es el caso más extremo y barato: solo necesitamos una muestra etiquetada.

Riesgo

Ese único ejemplo puede no ser representativo.

Idea clave

Pasar de 1-shot a 5-shot no garantiza mejorar siempre, pero normalmente reduce la dependencia de un único ejemplo concreto.

5-shot

La clase se representa usando cinco ejemplos.

Ventaja

El prototipo suele ser más estable porque promedia varias muestras.

Riesgo

Sigue habiendo pocos datos, así que la representación puede ser limitada.

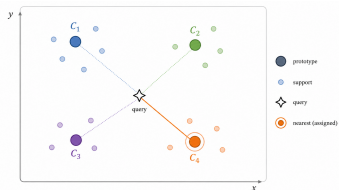
Clasificación por distancia

Idea principal

Una vez tenemos los prototipos, clasificamos cada imagen de consulta comparando su embedding con ellos.

Proceso

- obtenemos el embedding de la imagen query;
- calculamos su distancia a cada prototipo;
- asignamos la clase del prototipo más cercano.



La clase se decide por cercanía en el espacio de embeddings.

$$\hat{y} = \arg \min_k d(f(x), c_k)$$

Distancia euclídea: qué estamos midiendo

Distancia utilizada

En esta práctica usamos distancia euclídea para medir la cercanía entre un embedding y cada prototipo.

$$d(f(x), c_k) = \sqrt{\sum_{j=1}^d (f(x)_j - c_{k,j})^2}$$

Interpretación

Si una imagen está cerca del prototipo del 7, el sistema tenderá a clasificarla como 7.

Condición importante

Esto solo funciona bien si el espacio de embeddings agrupa cerca las imágenes semánticamente parecidas.

Tareas 1 y 2: del clasificador al extractor

Tarea 1

Preparar el mundo sin sietes

- cargar y normalizar MNIST;
- separar las imágenes del 7;
- entrenar con las clases conocidas;
- evaluar sobre clases conocidas.

Tarea 2

Construir el extractor

- tomar el modelo entrenado;
- elegir una capa intermedia;
- obtener embeddings;
- comprobar la dimensión del vector.

Qué debéis explicar

Por qué eliminamos el 7 y por qué usamos embeddings en lugar de píxeles.

Tarea 3: cálculo de prototipos

Idea principal

Para cada clase del **support set**, calculamos un **prototipo** como el vector medio de sus embeddings.

- Seleccionar los ejemplos de apoyo de una clase.
- Pasarlos por el extractor de características.
- Calcular el vector medio de sus embeddings.

Código básico

```
support_emb = feature_extractor.predict(X_support)

prototype = np.mean(
    support_emb,
    axis=0
)
```

Tarea 4: clasificación por distancia

Idea principal

Una imagen query se clasifica asignándola a la clase cuyo **prototipo está más cerca** en el espacio de embeddings.

- Obtener los embeddings de las imágenes query.
- Calcular la distancia a cada prototipo.
- Elegir la clase del prototipo más cercano.

Código básico

```
q_emb = feature_extractor.predict(X_query)

diff = q_emb[:, None, :] - prototypes[None, :, :]

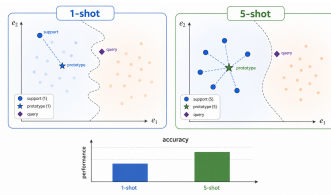
distances = np.linalg.norm(diff, axis=2)

predicted_classes = np.argmin(distances, axis=1)
```

Tarea 5: comparar 1-shot y 5-shot

Qué vais a hacer

- ejecutar la clasificación con prototipos 1-shot;
- ejecutar la clasificación con prototipos 5-shot;
- evaluar ambos casos sobre el mismo query set;
- guardar la accuracy de cada configuración;
- representar una gráfica de barras.



Comparación esperada entre usar 1 o 5 ejemplos.

Qué debéis interpretar

No basta con decir qué accuracy es mayor. Hay que explicar por qué 5 ejemplos pueden construir un prototipo más robusto.

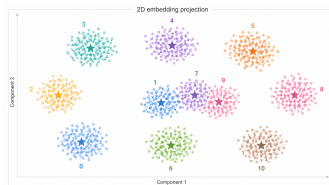
Tarea 6: visualizar el espacio de embeddings

Objetivo

Queremos interpretar si las imágenes quedan agrupadas de forma coherente en el espacio de características.

Qué vais a hacer

- obtener embeddings de imágenes de test;
- reducirlos a dos dimensiones con PCA o t-SNE;
- colorear los puntos por clase;
- marcar los prototipos, si es posible;
- observar dónde queda el número 7.



Proyección 2D del espacio de características.

Cómo interpretar los resultados

Qué buscamos

Un buen espacio de embeddings debería colocar cerca las imágenes de la misma clase y separar razonablemente clases diferentes.

Señales positivas

- agrupaciones claras por clase;
- prototipos cerca del centro de cada grupo;
- queries cercanas a su prototipo correcto;
- mejora razonable de 1-shot a 5-shot.

Señales problemáticas

- clases muy mezcladas;
- el 7 cerca de dígitos parecidos;
- prototipos desplazados;
- support set poco representativo.

Cómo interpretar los resultados

Qué buscamos

Un buen espacio de embeddings debería colocar cerca las imágenes de la misma clase y separar razonablemente clases diferentes.

Señales positivas

- agrupaciones claras por clase;
- prototipos cerca del centro de cada grupo;
- queries cercanas a su prototipo correcto;
- mejora razonable de 1-shot a 5-shot.

Señales problemáticas

- clases muy mezcladas;
- el 7 cerca de dígitos parecidos;
- prototipos desplazados;
- support set poco representativo.

Interpretación

Si el 7 queda cerca del 1 o del 9, algunas confusiones pueden ser razonables desde el punto de vista visual.

Qué puede salir mal

Few-shot Learning depende de varios factores

El método puede funcionar bien, pero no está garantizado.

1. Support set pobre

Los pocos ejemplos disponibles pueden no representar bien la clase nueva.

2. Embeddings malos

El extractor puede no separar bien la nueva clase en el espacio de características.

3. Clases parecidas

Algunos dígitos pueden quedar muy cerca porque son visualmente similares.

Qué puede salir mal

Few-shot Learning depende de varios factores

El método puede funcionar bien, pero no está garantizado.

1. Support set pobre

Los pocos ejemplos disponibles pueden no representar bien la clase nueva.

2. Embeddings malos

El extractor puede no separar bien la nueva clase en el espacio de características.

3. Clases parecidas

Algunos dígitos pueden quedar muy cerca porque son visualmente similares.

Mensaje importante

Few-shot Learning no elimina la necesidad de buenos datos y buenas representaciones. Solo cambia la forma de aprovecharlos.

Resumen final

Lo esencial

Few-shot Learning permite abordar situaciones en las que aparece una clase nueva con muy pocos ejemplos etiquetados.

1. Representar

Una red entrenada previamente convierte imágenes en embeddings.

2. Prototipar

Pocos ejemplos de apoyo permiten calcular un prototipo de clase.

3. Comparar

Las nuevas imágenes se clasifican por distancia al prototipo más cercano.

Mensaje final

El objetivo no es memorizar el 7 con pocos ejemplos, sino comprobar si el espacio de características aprendido permite reconocerlo por similitud.